AeroMind — Final PRD

Oct 21, 2026 · @Drone MatrX

Overview

AeroMind is a multi-agent AI system that autonomously analyzes heterogeneous drone data, diagnoses anomalies, evaluates mission risks, learns from historical missions, and generates actionable recommendations and reports through an intelligent orchestrator. It works with ArduPilot and Mission Planner, runs fully on a laptop with SITL, and is built in one day.

One-liner: Drop in a flight's files → get one page that says how the flight went, how the telemetry link held up the whole time, what went wrong and why, how risky the next mission is, and a corrected mission file to fly next.

Problem: After a flight, the useful information is spread across a telemetry log, a mission file, a parameter file and the pilot's memory. Reading it needs expertise, so most operators never learn why a flight drifted, drained the battery or lost link, and repeat the same mistakes.

Solution: Python tools turn every input into one set of verified metrics. An orchestrator then picks the right specialist agents, which write a plain-language flight summary, analyse telemetry signal quality, diagnose anomalies, score the next mission's risk using past flights, and propose fixes. A Verifier drops any claim the data does not support.

What makes it unique, while staying easy to build:

- **Whole-flight summary in one page:** a verdict, a phase-by-phase flight story, a telemetry signal timeline, and the top 3 improvements.
- **Telemetry link analysis:** dropouts located on the map (e.g. "link lost 12 s near WP5"), packet loss and RSSI where available.
- **Learns from history:** every flight is stored, so AeroMind compares each flight with your own baseline and predicts battery use for the next mission.
- **Closed loop with proof:** outputs a loadable `.waypoints` file; re-flying it shows the improvement in numbers.
- **Grounded and safe:** agents only see computed metrics, a Verifier checks every claim, and nothing is written to the drone automatically.

Goals and non-goals

The target for the day: a Streamlit dashboard where you upload one SITL flight and, in under a minute, see its summary, telemetry signal, anomalies, risk score, recommendations and fixed mission.

Goals

1. Ingest 4 data types (telemetry log, mission, parameters, pilot notes) into one metrics file.
2. Show a one-page flight summary: verdict, health score, flight story, telemetry signal, top 3 improvements.
3. Detect injected faults (wind, battery, link loss) in at least 3 of 4 scenarios.
4. Score next-mission risk using a history of at least 5 stored flights.
5. Generate a valid `mission_fixed.waypoints` and show a before/after comparison.
6. Run everything with two commands: one CLI, one dashboard.

Non-goals

- In-flight control or real-time alerts; AeroMind is post-flight.
- Writing parameters or missions to the drone automatically.
- Training ML models, PID auto-tuning, or real hardware flights.
- A Mission Planner C# plugin.

Input data

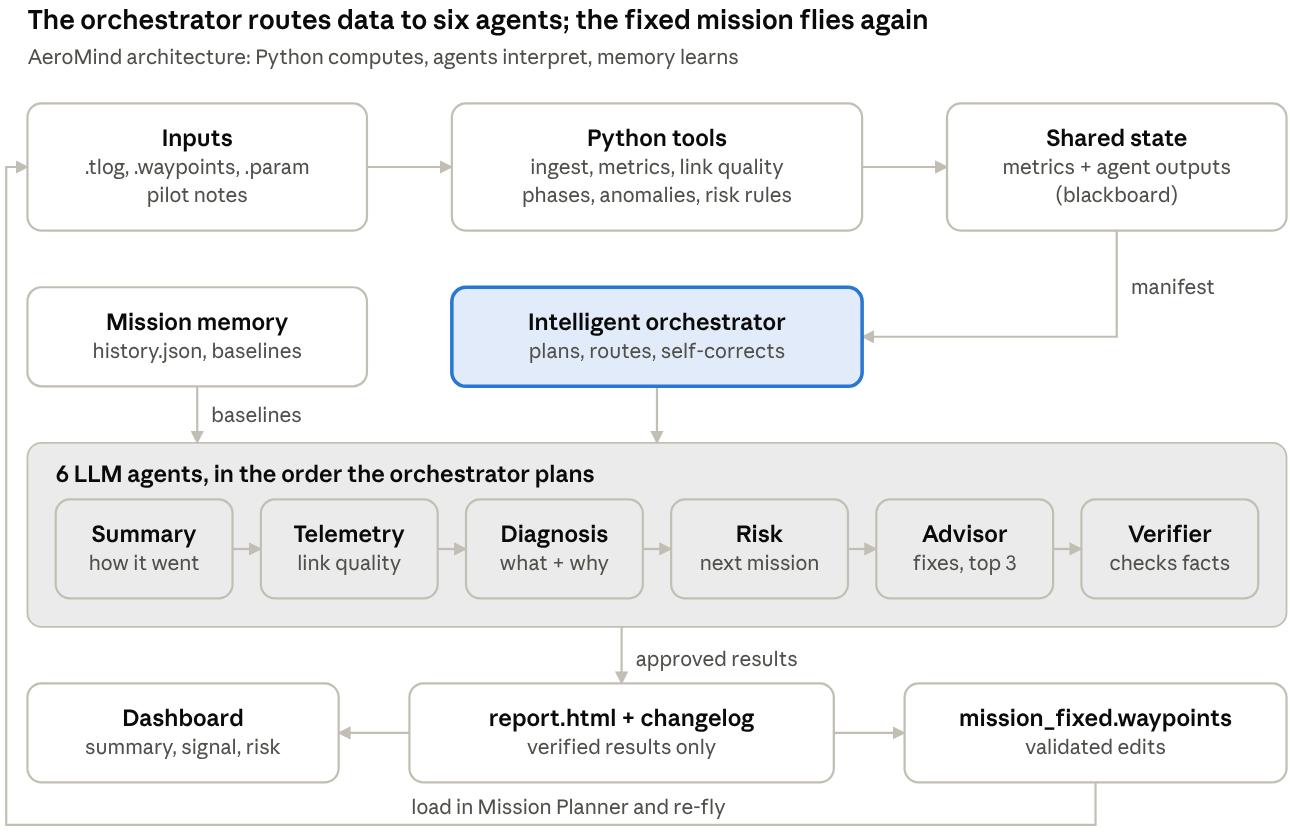
AeroMind reads five kinds of data, all of which Mission Planner already produces; only the telemetry log is required, and each extra input switches on more analysis.

Input	Format	How to get it	Required	What it unlocks
Telemetry log	`.tlog` (MAVLink stream)	Saved automatically by Mission Planner on every connection	Yes	All flight metrics, telemetry link quality, anomalies
Mission plan	`.waypoints`	Plan screen → Save WP File	Recommended	Planned vs actual track, mission fixes

Input	Format	How to get it	Required	What it unlocks
Parameters	`.param`	Config → Full Parameter List → Save	Optional	Failsafe and setup checks in the risk score
Pilot notes	Free text in the dashboard	Typed after the flight, e.g. "wobbly on landing"	Optional	Agents link what the pilot felt to the data
Flight history	`history.json` (built by AeroMind)	Appended after every analysis	Auto	Baselines, trends, battery prediction

P1: DataFlash `.bin` logs, which `pymavlink` reads through the same API, so the parser maps both into the same DataFrames.

Architecture



AeroMind architecture · tools, orchestrator, 6 agents, memory, outputs

Python turns every input into one shared state; the orchestrator reads its manifest and runs only the agents needed, which read memory baselines and add their outputs. Only Verifier-approved results reach the dashboard, report and fixed mission, and each analysed flight is saved back to memory.

Intelligent orchestrator

The orchestrator plans which agents to run from what data exists and what looks wrong, instead of running a fixed chain. It is about 60 lines of Python plus one short LLM call.

1. **Build a manifest:** which inputs were uploaded, which metrics exist, and each metric's status (good / caution / bad).
2. **Plan:** the LLM reads the manifest and returns an ordered list of agents with a one-line reason each. Python checks the plan against allowed agents and rules; if the call fails, a default plan runs.
3. **Run:** agents share one state dictionary (a blackboard); each reads what it needs and adds its output.
4. **Self-correct:** if the Verifier rejects more than 30% of an agent's claims, the orchestrator re-runs that agent once with the rejection reasons attached.

Situation in the manifest	Orchestrator decision
No mission file	Skip Mission Fixer; Diagnosis skips track checks
No Caution or Bad metrics	Skip root-cause analysis; Advisor gives maintenance tips only
Link metrics all good	Telemetry agent writes a one-line "link healthy" result
Fewer than 3 flights in history	Risk agent uses default rules and marks confidence as low
Pilot notes present	Pass notes to Flight Summary and Diagnosis
Always	Flight Summary first, Verifier last

The plan and reasons are shown in the dashboard, so viewers can see *why* each agent ran.

Agent specifications

Six LLM agents, each one prompt plus a JSON output schema, called through one shared helper `run_agent(name, system_prompt, state)` that uses JSON mode and retries once on bad output. The heavy lifting is done by six plain Python tools.

LLM agents

Agent	Answers	Reads	Writes
1. Flight Summary	How did the flight go?	metrics, phases, events, pilot notes	`verdict`, `flight_story[]`, `summary`
2. Telemetry Link	How was the signal across the whole flight?	link metrics, dropout list with positions	`link_findings[]`, `link_verdict`
3. Diagnosis	What went wrong, and why?	metrics, anomalies, pilot notes, cause hints	`findings[]`, `causes[]`
4. Risk	How risky is the next mission?	causes, params, history baselines, planned mission	`risk_level`, `risk_score`, `factors[]`, `predicted_battery_pct`
5. Advisor	What should change?	causes, risk factors, mission	`edits[]`, `recommendations[]` (top 3 ranked)
6. Verifier	Is every claim backed by data?	all outputs + metrics	`verdicts[]`, approved set

Python tools (no LLM)

Tool	Job
Ingestor	Reads `.tlog`, `.waypoints`, `.param`, notes into DataFrames
Metrics engine	Computes all metrics, statuses and the health score
Anomaly detector	Flags spikes with a rolling z-score (|z| > 3) on vibration, current, altitude error

Tool	Job
Phase splitter	Splits the flight into arm, takeoff, legs, RTL, land from mode changes and waypoint progress
Memory store	Saves each flight to `history.json`; returns baselines and trends
Mission I/O + Report	Applies approved edits, writes `.waypoints`, builds `report.html`

Prompt skeletons

- **Flight Summary:** "You are a drone flight-test engineer. Using only this JSON, give a verdict (Good / Needs attention / Unsafe), a short story per flight phase, and a 3-sentence summary a beginner can follow."
- **Telemetry Link:** "Describe link quality across the flight. For each dropout give time, duration and nearest waypoint. Say whether the link was safe for this mission."
- **Diagnosis:** "List findings citing metric names and values, then up to 3 root causes with confidence 0–1 and supporting finding ids. Use the hint rules. Use pilot notes as supporting evidence only."
- **Risk:** "Score next-mission risk 0–100 from these factors and history baselines. Give the top factors and predicted battery at landing. Mark confidence low if history has fewer than 3 flights."
- **Advisor:** "Propose mission edits from the allowed actions only, each tied to a cause id, then rank the top 3 improvements overall, including hardware and parameter advice."
- **Verifier:** "For each claim, check the cited metric exists, the value matches, and the threshold is crossed. Mark supported or rejected with a reason."

Flight summary: what the user sees

The first dashboard tab answers "how did the flight go, what was the signal like, and what should I improve" in one screen; the other tabs hold the detail.

Summary tab, top to bottom

1. **Verdict banner:** Good / Needs attention / Unsafe, the health score (0–100), and the 3-sentence summary.
2. **Key numbers row:** flight time, distance, max altitude, max speed, battery at landing, link uptime, max cross-track error.

3. **Flight story:** one line per phase with a status icon, e.g. *Takeoff — smooth, 40 m in 18 s · Leg 3 — drifted 6.8 m in 9 m/s wind · RTL — triggered by low battery at 18%*.
4. **Telemetry signal strip:** link quality across the whole flight as a coloured bar (green / amber / red), with dropouts marked and labelled by time and nearest waypoint.
5. **Top 3 improvements:** ranked, each with the reason and the evidence, e.g. *Reduce speed to 7 m/s on legs 2–4 (cross-track 6.8 m at 12 m/s, wind 9.1 m/s)*.
6. **Next-mission risk:** Low / Medium / High, the score, the top factors and predicted battery at landing.
7. **Downloads:** `report.html`, `mission_fixed.waypoints`, `changelog.md`.

Other tabs

Tab	Shows
Track	Planned vs actual path; path coloured by link quality; dropout points pinned
Telemetry	Link quality, packet loss and RSSI over time; heartbeat gaps; message rates
Anomalies	Each anomaly with time, value, plot snippet and linked cause
Power & attitude	Battery %, voltage, current, roll/pitch, vibration over time
History	This flight vs your baseline; trend charts across all stored flights
Agents	The orchestrator's plan, each agent's output, Verifier verdicts
Ask	Chat about this flight, answered from the metrics and agent outputs

Metrics and thresholds

Every metric comes from MAVLink messages already in the `.tlog`, extracted in one `pymavlink` pass; vibration bands follow ArduPilot guidance and the rest are project defaults to tune after a baseline flight. A missing message gives `null`, never a guess.

Flight metrics

Metric	Source	Good	Caution	Bad
Cross-track error, max per leg (m)	`NAV_CONTROLLER_OUTPUT.xtrack_error`, leg from `MISSION_CURRENT.seq`	< 1	1–3	> 3
Battery at landing (%)	`SYS_STATUS.battery_remaining`	> 30	20–30	< 20
Vibration max X/Y/Z (m/s²)	`VIBRATION.vibration_x/y/z`	< 30	30–60	> 60
Accelerometer clipping	`VIBRATION.clipping_0/1/2`	0	any increase	
Minimum satellites	`GPS_RAW_INT.satellites_visible`	⩾ 10	6–9	< 6
Maximum HDOP	`GPS_RAW_INT.eph` / 100	< 1.5	1.5–2.5	> 2.5
Maximum wind estimate (m/s)	`WIND.speed`	< 5	5–8	> 8
Maximum roll / pitch (°)	`ATTITUDE.roll/pitch`	< 25	25–35	> 35
Mission completion (%)	`MISSION_ITEM_REACHED.seq`	100		< 100
Warnings and failsafes	`STATUSTEXT` severity ⩽ 4, mode changes in `HEARTBEAT`	0	1–2	> 2
Time, distance, max altitude, max speed	timestamps, `GLOBAL_POSITION_INT`, `VFR_HUD`	info		

Telemetry link metrics (whole flight)

Metric	How computed	Good	Caution	Bad
Link uptime (%)	Share of flight time with no `HEARTBEAT` gap over 2 s	⩾ 99	95–99	< 95

Metric	How computed	Good	Caution	Bad
Dropouts	Each `HEARTBEAT` gap over 2 s: start time, duration, position, nearest waypoint	0	1–2, each ≤ 5 s	> 2, or any > 10 s
Packet loss (%)	Missing MAVLink sequence numbers (`msg.get_seq()`) from the vehicle	< 2	2–5	> 5
Radio RSSI, remote RSSI, noise	`RADIO_STATUS` (only with telemetry radios; absent in SITL)	trend + weakest 10% of flight flagged		
RC receiver RSSI	`RC_CHANNELS.rssi` (255 = not reported)	trend		
Position update rate (Hz)	`GLOBAL_POSITION_INT` arrivals per second	steady	drops > 50%	

Health score (0–100): start at 100, subtract 10 per Caution and 25 per Bad metric, floor at 0. Green ≥ 80, amber 50–79, red < 50.

Learning from historical missions

AeroMind "learns" by keeping a record of every flight and computing statistics from it; there is no model training, so it works from the third flight onward and is easy to explain in a viva.

Stored per flight in `history.json`: run id, date, scenario tag, mission length, duration, battery used, battery % per km, max cross-track error, link uptime, dropouts, max vibration, health score, causes found, edits applied, and the parent run id if this flight used a fixed mission.

Mechanism	How it works	Where it shows
Personal baselines	Mean and standard deviation of each metric over past flights; flag anything worse than mean + 2σ	"Worse than your usual" tags on the Summary and History tabs

Mechanism	How it works	Where it shows
Battery prediction	Median battery % used per km from past flights, applied to the next mission's length	Risk agent's predicted battery at landing
Fix effectiveness	Compares a re-flown mission with its parent run and records which edit improved which metric	Advisor's "what worked before" list
Recurring issues	A cause found in 3 or more of the last 5 flights is marked recurring	Raises the risk score; tops the improvements list

The demo scenarios plus their re-flights give about 8 stored flights, enough for every mechanism to show.

Risk scoring and mission fix rules

Python computes a base risk score from fixed rules so it is repeatable; the Risk agent may adjust it by at most ±10 points and must explain why.

Risk factor	Points
Predicted battery at landing < 20% / 20–30%	+30 / +15
Telemetry dropouts Bad / Caution	+20 / +10
GPS Bad / Caution	+20 / +10
Wind Bad in the latest flight	+15
Vibration Bad	+15
Each failsafe not set in `.param` (`FS_THR_ENABLE`, `BATT_FS_LOW_ACT`, `FENCE_ENABLE` = 0)	+10
Recurring issue from history	+10

Levels: Low < 30, Medium 30–59, High ≥ 60 (capped at 100).

Mission edits. The Advisor only proposes; Python checks each edit against these limits, applies it, and writes `mission_fixed.waypoints` and `changelog.md`. Rejected edits are listed with the reason.

Action	How Python applies it	Limits
`set_speed`	Inserts `DO_CHANGE_SPEED` (command 178, param2 = speed) before the waypoint	3–10 m/s
`change_altitude`	Edits ALT on a `NAV_WAYPOINT` row (command 16)	±20 m per edit, final 10–120 m
`remove_waypoint`	Deletes a `NAV_WAYPOINT` row and renumbers	Max 30% of waypoints; never home, takeoff (22), RTL (20), land (21)
`add_hold`	Sets param1 (hold seconds) on a `NAV_WAYPOINT` row	0–10 s

Global rules: at most 5 edits per run, each tied to a cause id; the mission must still end in RTL or LAND; 120 m matches the 400 ft green-zone limit in India's drone rules. A link-dropout cause produces advice (antenna placement, shorter range), not a mission edit.

Data contracts

All agents read and write one shared state object; fix these shapes first so every part can be built and tested on its own.

metrics.json (Metrics engine)

```
{
  "flight": {"duration_s": 412, "distance_m": 1850, "max_alt_m": 40.2,
"max_speed_ms": 12.1},
  "battery": {"remaining_end_pct": 18, "used_pct_per_km": 44.3},
  "tracking": {"xtrack_max_m": 6.8, "legs": [{"leg": 3, "xtrack_max_m": 6.8,
"speed_ms": 12.0}]},
  "vibration": {"max_x": 14, "max_y": 16, "max_z": 22, "clip_total": 0},
  "gps": {"min_sats": 12, "max_hdop": 0.9},
  "wind": {"max_ms": 9.1},
  "link": {"uptime_pct": 94.1, "packet_loss_pct": 3.2, "rssi": null,
           "dropouts": [{"t": 233, "duration_s": 12.4, "lat": 23.0231, "lon":
72.5702, "near_wp": 5}]},
  "phases": [{"name": "Leg 3", "t_start": 120, "t_end": 168}],
  "anomalies": [{"t": 141, "signal": "current", "z": 4.1}],
  "events": [{"t": 385, "text": "Battery low"}],
  "status": {"xtrack_max_m": "bad", "link_uptime": "bad", "battery_end":
"bad"},
  "health_score": 25
}
```


Orchestrator plan


```
{"plan": [
  {"agent": "flight_summary", "reason": "always first"},
  {"agent": "telemetry_link", "reason": "1 dropout of 12.4 s"},
  {"agent": "diagnosis", "reason": "3 bad metrics"},
  {"agent": "risk", "reason": "history has 6 flights"},
  {"agent": "advisor", "reason": "mission file present"},
  {"agent": "verifier", "reason": "always last"}]}
```


Agent outputs (examples)

```
{"flight_summary": {"verdict": "Needs attention",
   "flight_story": [{"phase": "Leg 3", "status": "bad", "text": "Drifted 6.8
m in 9.1 m/s wind"}]},
 "telemetry_link": {"link_verdict": "caution",
   "link_findings": [{"claim": "Link lost 12.4 s near WP5", "metric":
"link.dropouts[0].duration_s"}]},
 "diagnosis": {"causes": [{"id": "C1", "cause": "Crosswind drift at high
speed", "confidence": 0.8, "findings": ["F1", "F3"]}]},
 "risk": {"risk_level": "High", "risk_score": 65, "predicted_battery_pct":
16,
   "factors": ["battery < 20%", "telemetry dropout", "wind"]},
 "advisor": {"edits": [{"action": "set_speed", "from_wp": 1, "value_ms": 7,
"cause": "C1"}],
   "recommendations": ["Reduce speed to 7 m/s", "Raise telemetry antenna;
keep within 500 m", "Shorten route by 1 waypoint"]}}
```


Demo scenarios in Mission Planner SITL

Five short flights plus three re-flights (about 8 flights of 5–7 minutes) give you every log, a known fault in each, and enough history for the learning features.

Setup (about 30 minutes)

1. Mission Planner → **Simulation** tab → Multirotor; SITL downloads and starts automatically.
2. Plan an 8-waypoint loop at 40 m, about 1.5 km, ending in RTL. Save `mission.waypoints`; save parameters as `params.param`.
3. Set fault parameters in **Config → Full Parameter List** before arming; reset them after each flight.
4. Arm, switch to AUTO, let it finish. Mission Planner saves the `.tlog` in its `logs` folder.

ID	Scenario	How to create it	Expected detection	Expected output
S0	Baseline	No faults, speed 5 m/s	Verdict Good, health ≥ 80	No edits, Low risk
S1	Strong wind + fast flight	`SIM_WIND_SPD` 10, `SIM_WIND_DIR` 90, speed 12 m/s	Cross-track and wind Bad	`set_speed` 6–7 m/s

ID	Scenario	How to create it	Expected detection	Expected output
S2	Undersized battery	`BATT_CAPACITY` 1500 (mAh)	Battery at landing < 20%	Shorten route; High risk
S3	Telemetry dropout	Mid-mission, click **Disconnect** in Mission Planner for about 12 s, reconnect; repeat once	2 dropouts with time and nearest waypoint; uptime Bad	Link advice; risk raised
S4	Combined	S1 + S2 + one dropout	All three causes ranked	Speed + shorter route + link advice

For S3: with `FS_GCS_ENABLE` = 0 (the default) the drone keeps flying the mission while disconnected; if it is enabled, the drone will RTL, which AeroMind should also report as a failsafe event.

Mission Planner usually starts a new `.tlog` file on each reconnect, so the Ingestor accepts several `.tlog` files from one flight, merges them by timestamp, and treats the gap between files as a dropout.

Re-fly: load `mission_fixed.waypoints` for S1, S2 and S4 with the same faults, fly again, and run the comparison. Optional S5: weak GPS by lowering simulated satellites (search `SIM_GPS` in the parameter list; names vary by version).

Success metrics

The headline result is the before/after table: fixed missions measurably beat the originals under the same faults.

Metric	How measured	Target
Fault detection	Injected faults in S1–S4 flagged Caution or Bad	≥ 3 of 4
Root-cause accuracy	Top-ranked cause matches the injected fault	≥ 3 of 4
Dropout localisation	Detected dropout time within 3 s of when you disconnected	Both S3 dropouts
False alarms	Bad findings on baseline S0	0
Tracking improvement	Max cross-track error, S1 original vs fixed	≥ 40% lower

Metric	How measured	Target
Battery margin	Battery at landing, S2 fixed	⩾ 25%
Risk ranking	S0 scored Low; S2 and S4 scored High	All three correct
Battery prediction error	Predicted vs actual battery at landing on re-flights	Within 10 points
Grounding	Claims rejected by Verifier ÷ total claims	Report the rate
Speed	Files in → dashboard ready	< 60 s

Results table for the report: scenario, verdict, health before/after, risk before/after, key metric before/after, edits applied.

Tech stack, folder structure and run commands

Plain Python with no agent framework; each agent is a prompt and a schema, so the whole system installs with one `pip` line and runs with two commands. In the viva, mention that the orchestrator maps directly onto LangGraph or CrewAI.

Layer	Choice	Why
Simulator	Mission Planner built-in SITL	One click, no Linux setup
Parsing	`pymavlink`	Official MAVLink library; reads `.tlog` and `.bin`
Data	pandas, numpy	Metrics, z-scores, baselines
Charts	plotly	Interactive in Streamlit; map tiles with the open-street-map style
LLM	Any API with JSON mode, or local Ollama (e.g. Qwen 2.5 7B)	One config line to switch
Dashboard	Streamlit	Tabs, uploads, downloads, chat in about 150 lines
Report	Jinja2 → single HTML file	Print to PDF from the browser
Memory	`history.json`	No database to install

```
aeromind/
├── data/<scenario>/      # .tlog files, mission.waypoints, params.param
├── tools/
│   ├── ingest.py          # read + merge tlogs, mission, params, notes
│   ├── metrics.py         # flight + link metrics, statuses, health score
│   ├── anomalies.py       # rolling z-score
│   ├── phases.py          # phase splitter
│   ├── risk_rules.py      # base risk score
│   ├── memory.py          # history.json, baselines, trends
│   └── mission_io.py      # validate + apply edits, write .waypoints
├── agents/
│   ├── base.py            # run_agent()
│   ├── prompts.py         # 6 prompts + cause hints
│   └── orchestrator.py    # manifest -> plan -> run -> self-correct
├── report/template.html
├── run.py                 # CLI
├── app.py                 # Streamlit dashboard
├── history.json
└── outputs/<run_id>/      # state.json, report.html,
mission_fixed.waypoints, changelog.md
```

Run commands

```
pip install pymavlink pandas numpy plotly streamlit jinja2 python-dotenv

# analyse one flight (several .tlog files allowed)
python run.py --tlog data/s1/*.tlog --mission data/s1/mission.waypoints --
params data/params.param --notes "drifted on leg 3"

# analyse every scenario folder at once (fills history for the demo)
python run.py --batch data/

# before/after comparison
python run.py --compare outputs/s1 outputs/s1_fixed

# dashboard
streamlit run app.py
```

Demo safety: every agent output is cached in `outputs/<run_id>/state.json`; the dashboard's "Use cached run" option replays it, so the demo works even without internet or an LLM key.

One-day build plan

About 12 hours, with a working Summary tab by hour 8. Each step ends with a "done when" check; don't move on until it passes.

1. **Hour 0–1.5 · Collect data.** SITL setup; fly S0–S4; save mission and params. Done when: five scenario folders are filled.
2. **Hour 1.5–3.5 · Ingest + metrics.** Tlog merge, flight metrics, link metrics with dropouts, phases, z-score anomalies, health score. Done when: S3 shows 2 dropouts at the right times and S0 scores ⩾ 80.
3. **Hour 3.5–4 · Memory + risk rules.** `history.json`, baselines, battery % per km, base risk score. Done when: `--batch` fills history with 5 flights.
4. **Hour 4–6 · Agents + orchestrator.** Six prompts, `run_agent`, manifest, plan, Verifier re-run. Done when: S1 gives a wind/speed cause and a `set_speed` edit, and the plan skips Mission Fixer when no mission is given.
5. **Hour 6–7 · Mission I/O.** Validate, apply, write file and change log. Done when: `mission_fixed.waypoints` opens in Mission Planner.
6. **Hour 7–9 · Dashboard.** Summary tab first, then Track, Telemetry, History, Agents. Done when: uploading S4 shows verdict, story, signal strip, top 3 improvements and risk.

7. **Hour 9–10 · Re-fly + compare.** Fly fixed missions for S1, S2, S4; run `--compare`. Done when: the before/after table shows improvement.
8. **Hour 10–11 · Report + Ask tab.** HTML report export and flight chat. Done when: a question like "why did the link drop?" gets a cited answer.
9. **Hour 11–12 · Package.** Results table, README, 6-slide deck, 2-minute recording, cached demo runs.

Cut order if behind: Ask tab → HTML report (the dashboard is enough) → Power & attitude tab → Verifier re-run loop → S4. Never cut the Summary tab, telemetry analysis or the re-fly comparison.

Risks and mitigations

Risk	Mitigation
SITL has a perfect radio link, so `RADIO_STATUS` and RSSI are empty	Link analysis relies on heartbeat gaps and packet loss, which work in SITL; RSSI panels say "not reported" and light up on real-radio logs
A reconnect splits one flight into several `.tlog` files	Ingestor merges files by timestamp; the gap between files counts as a dropout
A message is missing (e.g. `WIND`)	Metric set to `null`; agents skip nulls; the dashboard shows "not available"
LLM returns invalid JSON	JSON mode plus one retry with the parse error; fall back to an empty result
LLM invents numbers	Agents see only computed metrics; Verifier rejects mismatched claims; risk base score is rule-based
Unsafe mission edit	Python bounds check rejects it before writing; listed in the change log
Internet or API fails during the demo	Replay cached runs from `state.json`
History too small for baselines	Batch-analyse all scenarios first; Risk marks confidence low under 3 flights

Risk	Mitigation
Fixed mission is not better	Report it honestly with the reason; still a valid result
SITL won't run on your laptop	Use public ArduPilot sample logs; demo the analysis without the re-fly

Deliverables, demo script and future scope

Deliverables

- [ ] GitHub repo with README: setup, run commands, architecture image
- [ ] Streamlit dashboard with cached runs for S0–S4 and re-flights
- [ ] Sample `report.html`, `mission_fixed.waypoints`, `changelog.md` per scenario
- [ ] Filled results table (before/after)
- [ ] 6-slide deck: problem, architecture, agents, live demo, results, future scope
- [ ] 2-minute demo video

Demo script (4 minutes)

1. Open the dashboard and upload the S4 files plus the note "felt it drifting on leg 3".
2. **Agents tab:** show the orchestrator's plan and why each agent was chosen.
3. **Summary tab:** verdict "Needs attention", the flight story, the telemetry strip with the dropout near WP5, and the top 3 improvements.
4. **Track tab:** planned vs actual path, coloured by link quality.
5. **History tab:** this flight vs your baseline; risk High with predicted battery at landing.
6. Download `mission_fixed.waypoints`, load it in Mission Planner, show the re-flown result and the before/after comparison.
7. **Ask tab:** type "why did the battery run low?" and show the cited answer.

Future scope

- Live mode that streams telemetry during flight and raises alerts.
- Pre-flight agent that checks a new mission and parameters before arming.
- RAG over ArduPilot docs for parameter-tuning suggestions.
- Real-drone logs with telemetry-radio RSSI and `.bin` DataFlash analysis.
- A Mission Planner plugin button that sends the latest log to AeroMind.